Introduction to Theoretical Computer Science

CS2490 - Session 26: More Push-down Automata

Pierre Cagne

Fall 2026

October 19, 2026

Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,

Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,

Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,

Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,

Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,

Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

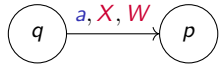
- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,
- a transition relation $\Delta : (\Sigma \cup \{\varepsilon\}) \times Q \times \Gamma \to \mathcal{P}_f(Q \times \Gamma^*)$.

Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,
- a transition relation $\Delta : (\Sigma \cup \{\varepsilon\}) \times Q \times \Gamma \to \mathcal{P}_f(Q \times \Gamma^*)$.

Graphically, $(p, W) \in \Delta(a, q, X)$ is represented as:

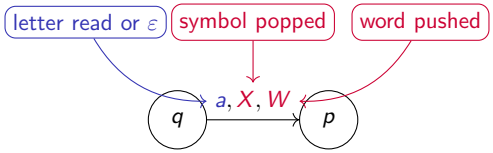


Pushdown automata

A pushdown automaton (PDA) over an input alphabet Σ is given by:

- a finite set of states Q,
- an initial state $q_0 \in Q$,
- a set of accepting states $F \subseteq Q$,
- a finite stack alphabet Γ,
- an initial stack symbol $z \in \Gamma$,
- a transition relation $\Delta : (\Sigma \cup \{\varepsilon\}) \times Q \times \Gamma \to \mathcal{P}_f(Q \times \Gamma^*)$.

Graphically, $(p, W) \in \Delta(a, q, X)$ is represented as:



Stack = memory

The PDA has a stack as memory.

Stack = memory

The PDA has a stack as memory. The stack is a word over the stack alphabet Γ: the first letter is the top of the stack, the last letter is the bottom of the stack.

Stack = memory

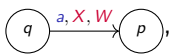
The PDA has a stack as memory. The stack is a word over the stack alphabet Γ: the first letter is the top of the stack, the last letter is the bottom of the stack.

Initially, the stack contains only the initial stack symbol z.

Stack = memory

The PDA has a stack as memory. The stack is a word over the stack alphabet Γ: the first letter is the top of the stack, the last letter is the bottom of the stack.

Initially, the stack contains only the initial stack symbol z.

At each transition 

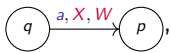
, the PDA in state q:

- pops the top symbol X from the stack,
- reads a and moves to the new state p,
- and pushes W onto the stack.

Stack = memory

The PDA has a stack as memory. The stack is a word over the stack alphabet Γ: the first letter is the top of the stack, the last letter is the bottom of the stack.

Initially, the stack contains only the initial stack symbol z.

At each transition 

, the PDA in state q:

- pops the top symbol X from the stack,
- reads a and moves to the new state p,
- and pushes W onto the stack.

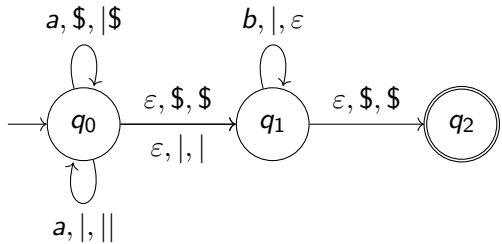
After the transition, the new top of the stack is the first symbol of W.

Example

Recall the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.

Example

Recall the PDA we designed for $L = \{a^n b^n \mid n \geq 0\}$.



Example (cont'd)

Let's run this PDA on input *aaaaabbbbb*.

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

A configuration (or instantaneous description) of $\mathcal{A}$ is a triple (q, w, S) where:

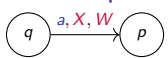
- q is a state,
- w is a word,
- S is a stack.

Configurations

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

A configuration (or instantaneous description) of $\mathcal{A}$ is a triple (q, w, S) where:

- q is a state,
- w is a word,
- S is a stack.

A computation step is written $(q, w, S) \vdash (p, u, T)$. It is only possible if there is a transition 

and $w = au$, $S = XS'$, and $T = WS'$.

Computations

A computation of $\mathcal{A}$ on a word w is a sequence of computation steps from the initial configuration (q_0, w, z) to any configuration (q, ε, S):

$$(q_0, w_1 w_2 \dots w_n, z) \vdash (q_1, w_2 \dots w_n, S_1) \vdash \dots \vdash (q, \varepsilon, S)$$

Computations

A computation of $\mathcal{A}$ on a word w is a sequence of computation steps from the initial configuration (q_0, w, z) to any configuration (q, ε, S):

$$(q_0, w_1 w_2 \dots w_n, z) \vdash (q_1, w_2 \dots w_n, S_1) \vdash \dots \vdash (q, \varepsilon, S) \qquad (\text{i.e.}, (q_0, w, z) \vdash^* (q, \varepsilon, S))$$

Computations

A computation of $\mathcal{A}$ on a word w is a sequence of computation steps from the initial configuration (q_0, w, z) to any configuration (q, ε, S):

$$(q_0, w_1 w_2 \dots w_n, z) \vdash (q_1, w_2 \dots w_n, S_1) \vdash \dots \vdash (q, \varepsilon, S) \qquad (\text{i.e.,} (q_0, w, z) \vdash^* (q, \varepsilon, S))$$

The computation is successful if $q \in F$. We then say that $\mathcal{A}$ accepts w.

Computations

A computation of $\mathcal{A}$ on a word w is a sequence of computation steps from the initial configuration (q_0, w, z) to any configuration (q, ε, S):

$$(q_0, w_1 w_2 \dots w_n, z) \vdash (q_1, w_2 \dots w_n, S_1) \vdash \dots \vdash (q, \varepsilon, S) \qquad (\text{i.e.,} (q_0, w, z) \vdash^* (q, \varepsilon, S))$$

The computation is successful if $q \in F$. We then say that $\mathcal{A}$ accepts w.
Note: the content of S at the end of the computation does not matter for acceptance.

Computations

A computation of $\mathcal{A}$ on a word w is a sequence of computation steps from the initial configuration (q_0, w, z) to any configuration (q, ε, S):

$$(q_0, w_1 w_2 \dots w_n, z) \vdash (q_1, w_2 \dots w_n, S_1) \vdash \dots \vdash (q, \varepsilon, S) \qquad (\text{i.e.,} (q_0, w, z) \vdash^* (q, \varepsilon, S))$$

The computation is successful if $q \in F$. We then say that $\mathcal{A}$ accepts w.
Note: the content of S at the end of the computation does not matter for acceptance.

The language recognized by $\mathcal{A}$ is the set of all the words it accepts:

$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \exists S, \exists q \in F, (q_0, w, z) \vdash^* (q, \varepsilon, S)\}$$

Practice!

Let's design a PDA that recognizes the language of binary words with the same number of 0s and 1s.

Practice!

Let's design a PDA that recognizes the language of binary words with the same number of 0s and 1s.

The idea: when reading a 0, either it corresponds to a previously read 1 (the stack will tell us), or it will need to be matched with a future 1 (leave a clue on the stack). The same goes for 1s.

Practice!

Let's design a PDA that recognizes the language of binary words with the same number of 0s and 1s.

The idea: when reading a 0, either it corresponds to a previously read 1 (the stack will tell us), or it will need to be matched with a future 1 (leave a clue on the stack). The same goes for 1s.

Thus, beside the initial stack symbol \$, two other stack symbols:

Practice!

Let's design a PDA that recognizes the language of binary words with the same number of 0s and 1s.

The idea: when reading a 0, either it corresponds to a previously read 1 (the stack will tell us), or it will need to be matched with a future 1 (leave a clue on the stack). The same goes for 1s.

Thus, beside the initial stack symbol \$, two other stack symbols:

- $|_0$ to remember an unmatched 0,
- $|_1$ to remember an unmatched 1.

Practice!

Let's design a PDA that recognizes the language of binary words with the same number of 0s and 1s.

The idea: when reading a 0, either it corresponds to a previously read 1 (the stack will tell us), or it will need to be matched with a future 1 (leave a clue on the stack). The same goes for 1s.

Thus, beside the initial stack symbol \$, two other stack symbols:

- $|_0$ to remember an unmatched 0,
- $|_1$ to remember an unmatched 1.

How many states?

Practice!

Let's design a PDA that recognizes the language of binary words with the same number of 0s and 1s.

The idea: when reading a 0, either it corresponds to a previously read 1 (the stack will tell us), or it will need to be matched with a future 1 (leave a clue on the stack). The same goes for 1s.

Thus, beside the initial stack symbol \$, two other stack symbols:

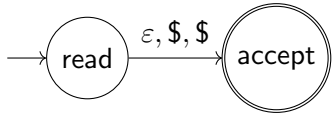
- $|_0$ to remember an unmatched 0,
- $|_1$ to remember an unmatched 1.

How many states? Two are enough: one to read the input, one to accept.

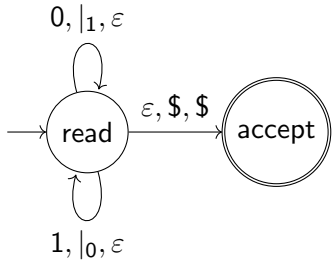
Practice! (cont'd)



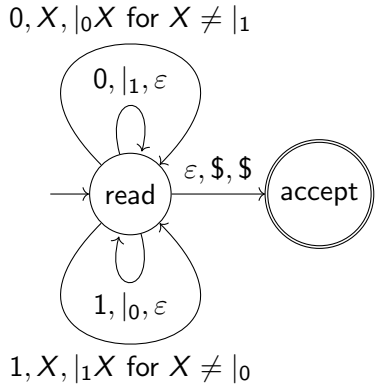
Practice! (cont'd)



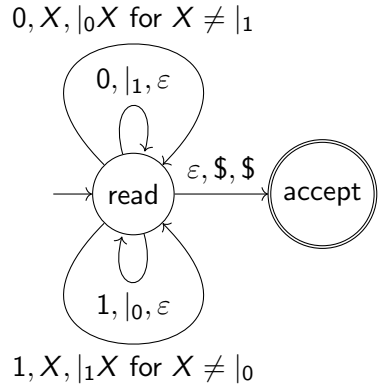
Practice! (cont'd)



Practice! (cont'd)



Practice! (cont'd)



At any point, the stack contains only $|_0$s or only $|_1$s (plus \$ at the bottom).

More practice!

Let's design a PDA that recognizes the language of palindromes over the alphabet $\{a, b, c\}$.

More practice!

Let's design a PDA that recognizes the language of palindromes over the alphabet $\{a, b, c\}$.

The idea: when reading a symbol, either we are in the first half of the word (push clues onto the stack), or we are in the second half of the word (consume the clues from the stack).

More practice!

Let's design a PDA that recognizes the language of palindromes over the alphabet $\{a, b, c\}$.

The idea: when reading a symbol, either we are in the first half of the word (push clues onto the stack), or we are in the second half of the word (consume the clues from the stack).

Thus, beside the initial stack symbol \$, one stack symbol per letter:

More practice!

Let's design a PDA that recognizes the language of palindromes over the alphabet $\{a, b, c\}$.

The idea: when reading a symbol, either we are in the first half of the word (push clues onto the stack), or we are in the second half of the word (consume the clues from the stack).

Thus, beside the initial stack symbol \$, one stack symbol per letter:

- A for a,
- B for b,
- C for c.

More practice!

Let's design a PDA that recognizes the language of palindromes over the alphabet $\{a, b, c\}$.

The idea: when reading a symbol, either we are in the first half of the word (push clues onto the stack), or we are in the second half of the word (consume the clues from the stack).

Thus, beside the initial stack symbol \$, one stack symbol per letter:

- A for a,
- B for b,
- C for c.

How many states?

More practice!

Let's design a PDA that recognizes the language of palindromes over the alphabet $\{a, b, c\}$.

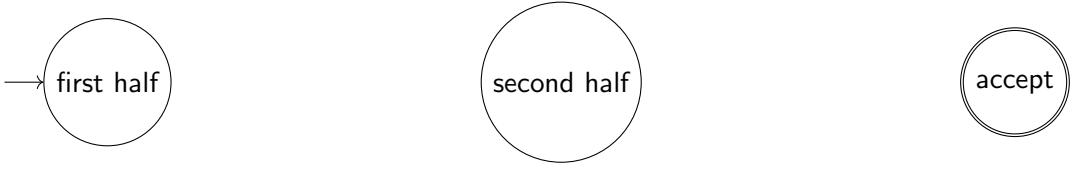
The idea: when reading a symbol, either we are in the first half of the word (push clues onto the stack), or we are in the second half of the word (consume the clues from the stack).

Thus, beside the initial stack symbol \$, one stack symbol per letter:

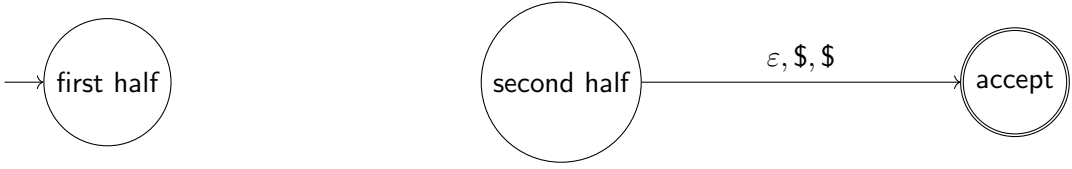
- A for a,
- B for b,
- C for c.

How many states? Three should do it: one to read the first half, one to read the second half, one to accept.

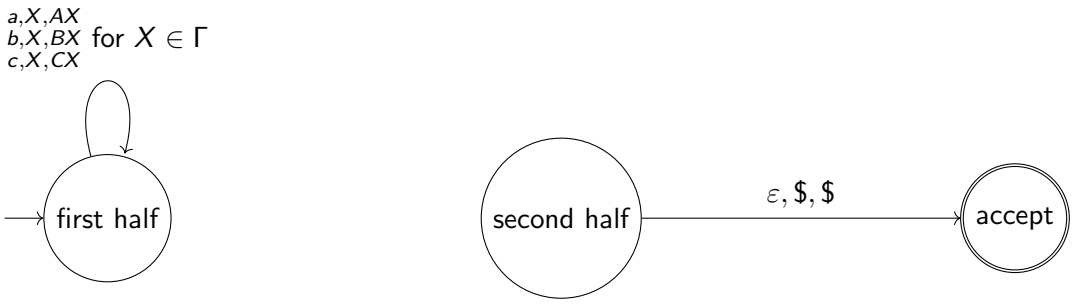
More practice! (cont'd)



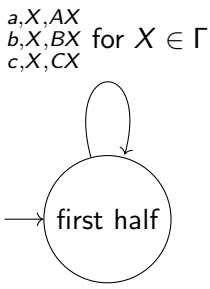
More practice! (cont'd)

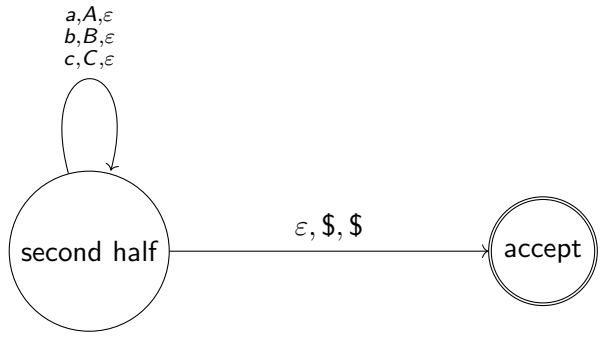


More practice! (cont'd)

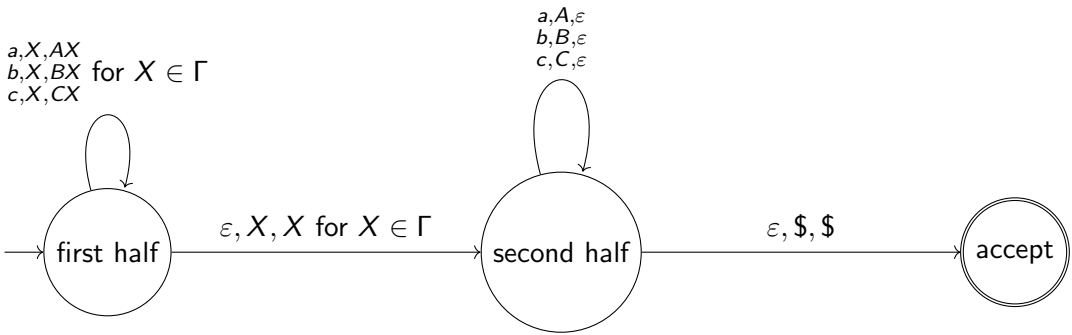


More practice! (cont'd)

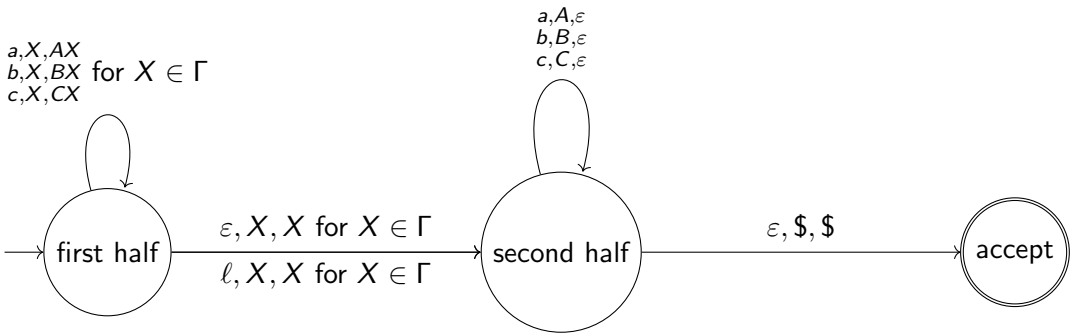




More practice! (cont'd)



More practice! (cont'd)



More practice! (cont'd)

Wait!

More practice! (cont'd)

Wait! How are we sure the computation switches from the first half to the second half at the right moment?

More practice! (cont'd)

Wait! How are we sure the computation switches from the first half to the second half at the right moment?

We aren't! That's the beauty of non-determinism: one of the possible computations will jump at the right time, and that all that matters.

More practice! (cont'd)

Wait! How are we sure the computation switches from the first half to the second half at the right moment?

We aren't! That's the beauty of non-determinism: one of the possible computations will jump at the right time, and that all that matters.

Let's see it in action on the word abbcabacbba.

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading a a, push a clue onto the stack; when reading a b, pop a clue from the stack every three bs.

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading a a, push a clue onto the stack; when reading a b, pop a clue from the stack every three bs.

Thus, beside the initial stack symbol \$, one stack symbol $|$.

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading a a, push a clue onto the stack; when reading a b, pop a clue from the stack every three bs.

Thus, beside the initial stack symbol \$, one stack symbol $|$.

How many states?

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading a a, push a clue onto the stack; when reading a b, pop a clue from the stack every three bs.

Thus, beside the initial stack symbol \$, one stack symbol $|$.

How many states? One to read all the as

One more!

Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading a a, push a clue onto the stack; when reading a b, pop a clue from the stack every three bs.

Thus, beside the initial stack symbol \$, one stack symbol $|$.

How many states? One to read all the as and three to read the bs.

One more!

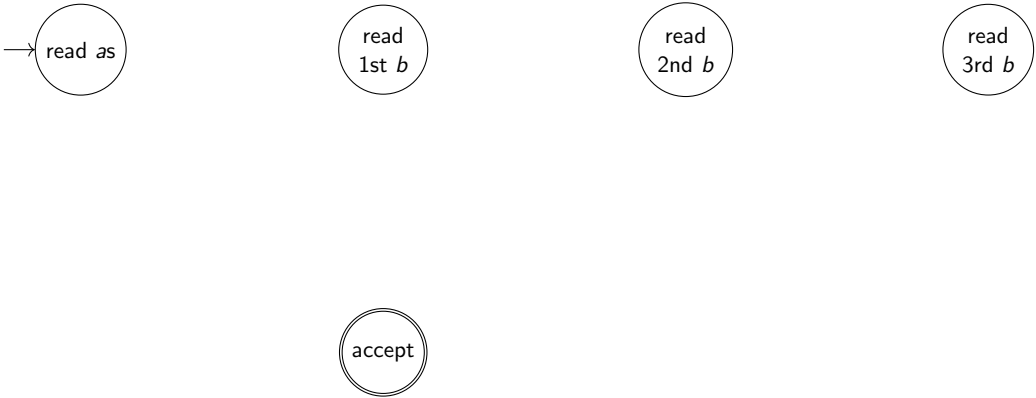
Let's design a PDA that recognizes the language $L = \{a^n b^{3n} \mid n \geq 0\}$.

The idea: when reading a a, push a clue onto the stack; when reading a b, pop a clue from the stack every three bs.

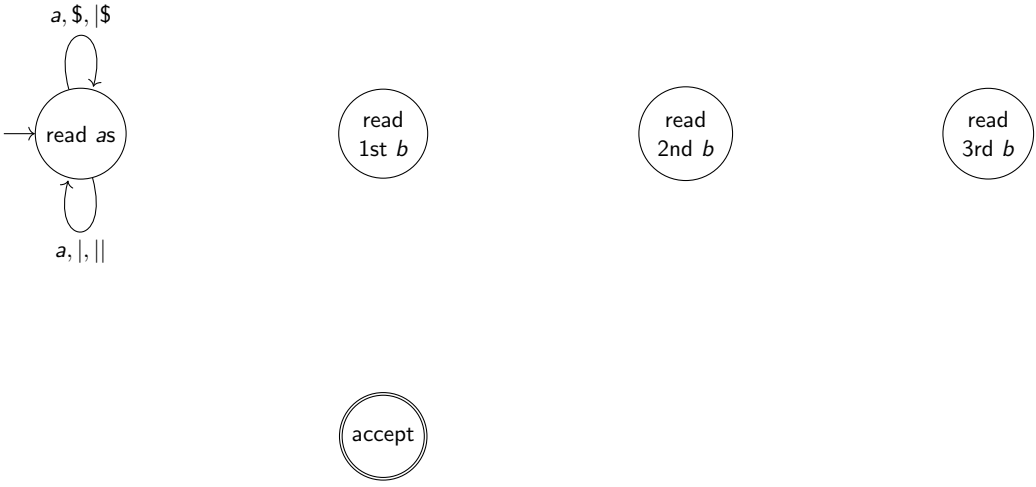
Thus, beside the initial stack symbol \$, one stack symbol $|$.

How many states? One to read all the as and three to read the bs. Plus one to accept.

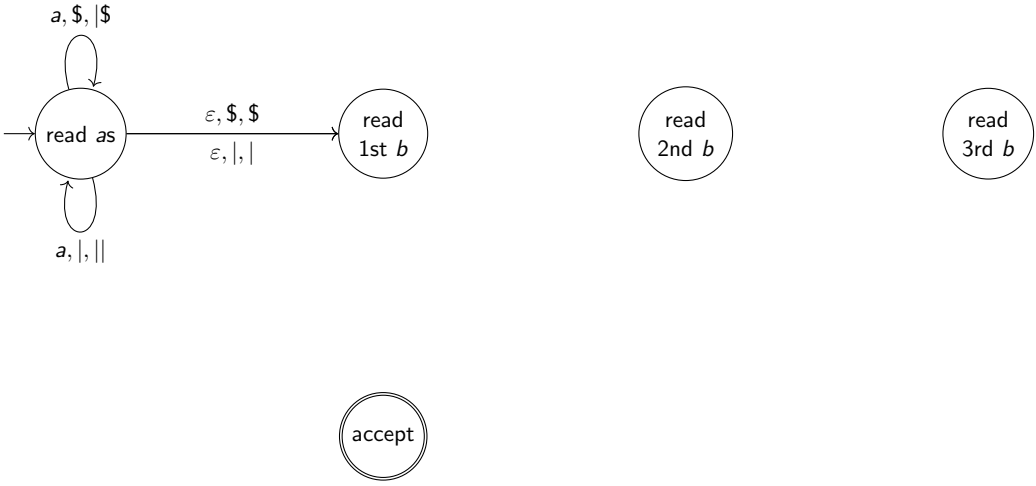
One more! (cont'd)



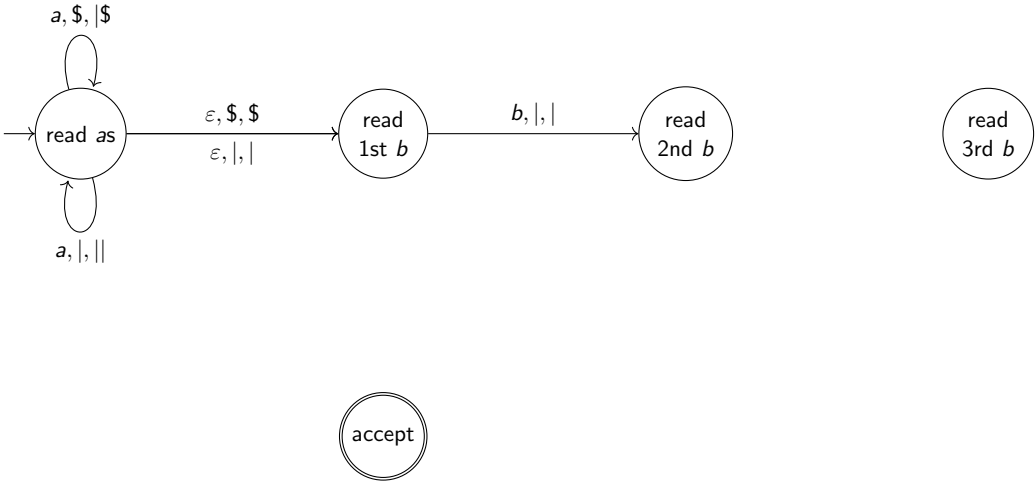
One more! (cont'd)



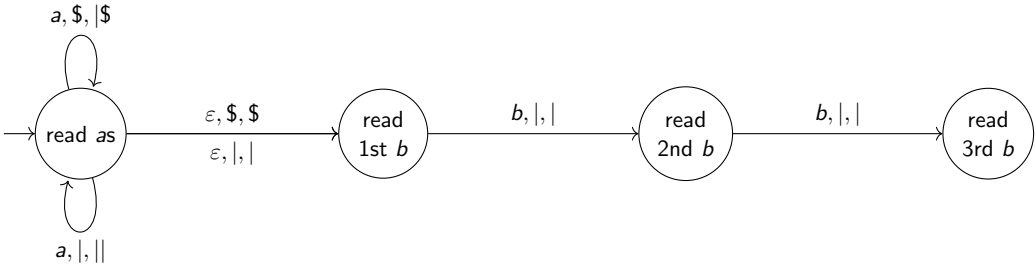
One more! (cont'd)



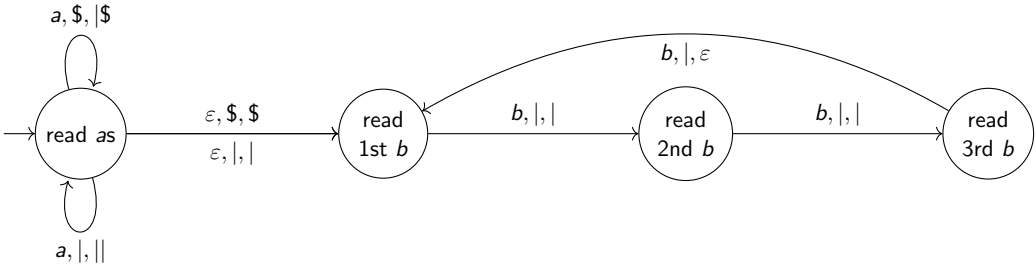
One more! (cont'd)



One more! (cont'd)

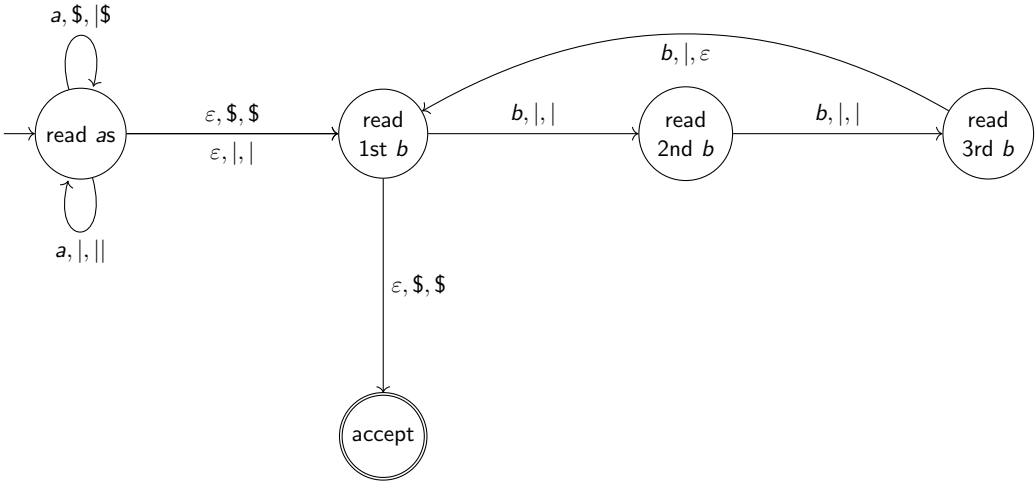


One more! (cont'd)

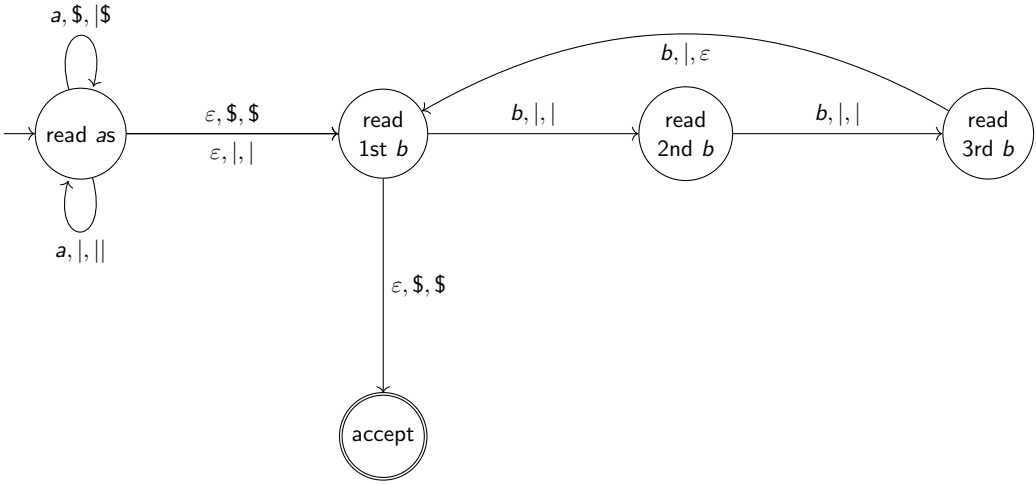




One more! (cont'd)



One more! (cont'd)



Let's see it in action on the word aaabbbbbbbbb.